

Design-Dokument

Geschwindigkeitsabhängiges Schutzsystem

Safety- und Security-Demonstrator mit rotierendem Aktor

Gruppe 3

Andreas Günter Kostecka
Jonas Gök
Loïc Thill
David Freudenthaler

FH Campus Wien

Stand: 17. Oktober 2026

Inhaltsverzeichnis

1 Projektbeschreibung **2**

1.1 Ziel und Systemidee 2

1.2 Systemaufbau 2

1.3 Funktionsablauf 3

1.4 Zentrale Designparameter 3

1.5 Grenzen des Demonstrators 3

2 Softwarearchitektur und Design **4**

2.1 Raspberry Pi 1 – Host und Bedienung 4

2.2 Raspberry Pi 2 – Steuerung und Überwachung 4

2.3 Safety-Logik-Thread 4

2.4 Gemeinsam verwendete Daten 5

2.5 Bluetooth-Kommunikation 5

2.6 Motorsteuerung 5

2.7 Initialisierung und kontrolliertes Beenden 6

2.8 Programmierrichtlinien und Versionskontrolle 6

3 Programmablauf und Safety-Logik **7**

4 Safety Hazards **8**

4.1 Übersicht 8

4.2 Adressierte Hazards 9

4.3 Nicht vollständig adressierte Hazards 10

5 Security-Betrachtung **11**

5.1 Relevante Bedrohungen 12

5.2 Abgrenzung 12

6 Anforderungen und Tests **12**

6.1 Safety-Anforderungen 13

6.2 Security-Anforderungen 13

6.3 Unterstützende Anforderungen 13

6.4 Geplante Tests 14

1 Projektbeschreibung

1.1 Ziel und Systemidee

Ziel des Projekts ist ein Demonstrator für ein geschwindigkeitsabhängiges Schutzsystem eines rotierenden Aktors. Das Prinzip orientiert sich vereinfacht an *Speed and Separation Monitoring* (SSM): Mit steigender Geschwindigkeitsstufe wird ein größerer Sicherheitsabstand verwendet. Unterschreitet ein Objekt diesen Abstand, wird die Motoransteuerung abgeschaltet.

Der Demonstrator dient ausschließlich zur Untersuchung eines sicherheitsgerichteten Prinzips. Er ist kein zertifiziertes Safety-System. Insbesondere werden Raspberry Pi 4B, HC-SR04, Bluetooth und der verwendete Motortreiber nicht als Safety-zertifizierte Komponenten betrachtet.

Sicherer Zustand: Motor AUS – der Rotor wird nicht aktiv angetrieben.

Ein Software-Stopp bedeutet nicht automatisch mechanischen Stillstand. Der Rotor kann aufgrund seiner Massenträgheit nachlaufen. Diese Nachlaufzeit muss bei der Festlegung der Sicherheitsabstände berücksichtigt werden.

1.2 Systemaufbau

Das System besteht aus zwei Raspberry Pi 4B, zwei Ultraschall-Abstandssensoren HC-SR04, einem Elektromotor mit Rotor, einem Motortreiber, einem LCD-Display und einer Bluetooth-Verbindung zwischen den Raspberry Pis.

Komponente	**Aufgabe**	**Bedeutung für Safety**
Raspberry Pi 1	Bedienung, Auswahl der Geschwindigkeitsstufe, LCD-Anzeige, Logging und Bluetooth-Kommunikation	Liefert Sollwerte und zeigt Zustände an. LCD und Logging sind keine Safety-Funktionen.
Raspberry Pi 2	Sensorerfassung, Plausibilitätsprüfung, Safety-Logik und Motorsteuerung	Führt die lokale Safety-Entscheidung aus und kann den Motor unabhängig von Pi 1 abschalten.
2 × HC-SR04	Abstandsmessung zum Gefahrenbereich	Zwei gleiche Sensoren ermöglichen einen Quervergleich, bilden aber keine nachgewiesene Safety-Redundanz.
Motortreiber / Motor	Umsetzung der Motoransteuerung und rotierende Gefahrenquelle	Motor AUS entspricht PWM = 0 und deaktiviertem Enable-Signal. Mechanischer Stillstand ist dadurch nicht garantiert.
Bluetooth	Bidirektionale Kommunikation zwischen Pi 1 und Pi 2	Wird mit Heartbeat, Timeout und Plausibilitätsprüfungen überwacht.

Tabelle 1: Komponenten und Aufgaben des Demonstrators

1.3 Funktionsablauf

Nach dem Start initialisieren beide Raspberry Pis ihre benötigten Schnittstellen und Threads. Raspberry Pi 2 hält die Motoransteuerung während der Initialisierung im sicheren Zustand. Erst wenn gültige Sensordaten, eine gültige Kommunikation und eine gültige Geschwindigkeitsstufe vorliegen, darf der Motor freigegeben werden.

Raspberry Pi 1 sendet die gewünschte Geschwindigkeitsstufe `LOW`, `MEDIUM` oder `HIGH` sowie einen Heartbeat an Raspberry Pi 2. Raspberry Pi 2 liest beide Abstandssensoren aus, prüft die Messwerte und wählt den zugehörigen Sicherheitsabstand. Befindet sich ein Objekt innerhalb dieses Abstands oder schlägt eine Safety-Prüfung fehl, wird der Motor abgeschaltet.

Raspberry Pi 2 sendet Status, Abstandswerte und Fehlermeldungen an Raspberry Pi 1 zurück. Diese Informationen werden am LCD angezeigt und können protokolliert werden.

1.4 Zentrale Designparameter

Die folgenden Werte sind Projektparameter für den Demonstrator und müssen am realen Aufbau getestet und gegebenenfalls angepasst werden.

Parameter	**Startwert**	**Bedeutung**
Heartbeat Pi 1 → Pi 2	100 ms (10 Hz)	Zyklische Lebensmeldung und Übertragung der Sollstufe
Kommunikations-Timeout	300 ms	Keine gültige Nachricht innerhalb dieses Zeitraums führt zu Motor AUS
Safety-Logik-Zyklus	50 ms	Zyklische Prüfung der Safety-Bedingungen auf Pi 2
Max. Alter der Sensordaten	150 ms	Ältere Messwerte dürfen nicht für eine Freigabe verwendet werden
Sensorabweichung	$>$ 5 cm in zwei aufeinanderfolgenden Messzyklen	Sensorwerte werden als unplausibel bewertet
Sicherheitsabstände	$S_{\text{LOW}} < S_{\text{MEDIUM}} < S_{\text{HIGH}}$	Endgültige Werte werden am Demonstrator anhand von Reaktions- und Nachlaufzeit festgelegt

Tabelle 2: Zentrale Startwerte des Designs

1.5 Grenzen des Demonstrators

Die tatsächliche Motordrehzahl wird nicht mit einem Encoder gemessen. Daher darf nicht angenommen werden, dass ein PWM-Wert proportional zur realen Drehzahl ist. Die Geschwindigkeitsstufen stellen lediglich definierte Ansteuerungsklassen dar. Außerdem können zwei gleiche HC-SR04 durch eine gemeinsame Fehlerursache ähnliche falsche Werte liefern. Ein Defekt des Motortreibers, bei dem der Motor trotz Abschaltsignal weiter bestromt wird, kann mit der vorhandenen Architektur nicht sicher beherrscht werden.

2 Softwarearchitektur und Design

Die Software wird in C unter Linux auf beiden Raspberry Pis implementiert. Threads werden verwendet, um Sensorerfassung, Kommunikation, Bedienung und Safety-Logik voneinander zu trennen. Safety-relevante Entscheidungen werden auf Raspberry Pi 2 getroffen; ein Ausfall von LCD oder Logging darf die Motorabschaltung nicht verhindern.

2.1 Raspberry Pi 1 – Host und Bedienung

Modul / Thread	**Aufgabe**
Initialisierung	Bluetooth, LCD, Logging und benötigte Threads starten
UI / Eingabe	Geschwindigkeitsstufe `LOW`, `MEDIUM`, `HIGH` auswählen und Programmende anfordern
Kommunikations-/Heartbeat-Thread	Sollstufe und Heartbeat alle 100 ms senden; Status und Fehlermeldungen von Pi 2 empfangen
Anzeige	Systemzustand, Abstände, Motorgeschwindigkeit und Fehler am LCD anzeigen
Logging	Status- und Fehlerereignisse protokollieren; keine Safety-Funktion

Tabelle 3: Softwareaufgaben auf Raspberry Pi 1

2.2 Raspberry Pi 2 – Steuerung und Überwachung

Modul / Thread	**Aufgabe**
Initialisierung	Motor sicher deaktivieren, Sensoren, Bluetooth und Threads initialisieren
Sensor-Thread 1 / 2	HC-SR04 auslesen und Messwert, Gültigkeitsstatus und Zeitstempel bereitstellen; Sensoren zeitlich versetzt triggern
Kommunikations-Thread	Nachrichten von Pi 1 empfangen und prüfen; Heartbeat-Timeout überwachen; Status an Pi 1 senden
Safety-Logik-Thread	Zyklisch alle 50 ms Sensorplausibilität, Kommunikation und Datenaktualität prüfen; Sicherheitsabstand auswählen und Motorfreigabe entscheiden
Motorsteuerung	Motorstufe setzen oder Motor mit PWM = 0 und Enable = OFF abschalten

Tabelle 4: Softwareaufgaben auf Raspberry Pi 2

2.3 Safety-Logik-Thread

Der Safety-Logik-Thread ist ein eigener Thread auf Raspberry Pi 2. Er führt keine blockierenden Netzwerk- oder Logging-Aufrufe aus. In jedem Zyklus werden folgende Prüfungen in dieser Reihenfolge ausgeführt:

1. Sind beide Sensorwerte gültig und zueinander plausibel?

2. Ist die Kommunikation aktuell und der empfangene Befehl gültig?
3. Sind die verwendeten Sensordaten noch aktuell?
4. Welcher Sicherheitsabstand gehört zur gültigen Geschwindigkeitsstufe?
5. Liegt der aktuelle Abstand innerhalb des Sicherheitsabstands?

Schlägt eine der ersten drei Prüfungen fehl, wird ein Fehlerzustand gesetzt und die Motoransteuerung deaktiviert. Erst wenn alle Prüfungen erfolgreich sind, darf die normale Motorsteuerung ausgeführt werden.

2.4 Gemeinsam verwendete Daten

Sensor- und Kommunikations-Threads aktualisieren Daten, die vom Safety-Logik-Thread gelesen werden. Damit keine Race Conditions entstehen, werden zusammengehörige Werte als Datensatz behandelt, zum Beispiel:

```
struct sensor_data {
    float distance_cm;
    bool valid;
    struct timespec timestamp;
};
```

Der schreibende Thread aktualisiert den vollständigen Datensatz geschützt durch einen Mutex. Der Safety-Logik-Thread kopiert den Datensatz unter demselben Mutex in eine lokale Variable und arbeitet anschließend nur mit diesem Snapshot. Zeitmessungen für Timeouts und Datenalter sollen mit einer monotonen Uhr erfolgen.

2.5 Bluetooth-Kommunikation

Die Kommunikation ist bidirektional. Raspberry Pi 1 sendet Sollstufe, Heartbeat, Sequenznummer und gegebenenfalls einen Stop/Shutdown-Befehl. Raspberry Pi 2 sendet Status, aktuelle Abstandswerte und Fehlermeldungen zurück.

Nachrichten werden mindestens auf Nachrichtentyp, Länge, Wertebereich und Sequenznummer geprüft. Alte, doppelte oder unzulässige Befehle werden nicht als gültige Motorfreigabe verwendet. Bleibt eine gültige Nachricht länger als 300 ms aus, wird die Kommunikation als fehlerhaft bewertet und der Motor abgeschaltet.

2.6 Motorsteuerung

Die Motorsteuerung erhält ausschließlich eine bereits durch die Safety-Logik freigegebene Geschwindigkeitsstufe. Eine unbekannte oder ungültige Stufe führt nicht zu einem Default-Betrieb, sondern zu Motor AUS. Der sichere Softwarezustand wird durch

`PWM = 0` und `Enable = OFF`

angefordert. Der mechanische Nachlauf des Rotors wird dadurch nicht aufgehoben.

2.7 Initialisierung und kontrolliertes Beenden

Beim Start von Raspberry Pi 2 ist die Motoransteuerung deaktiviert. Eine Motorfreigabe ist erst erlaubt, wenn Initialisierung, Sensorprüfung und Kommunikationsprüfung erfolgreich abgeschlossen wurden.

Fordert der Benutzer auf Raspberry Pi 1 das Programmende an, sendet Pi 1 einen Stop/Shutdown-Befehl. Pi 2 setzt zuerst den Motor in den sicheren Zustand, beendet danach seine Threads und schließt die Bluetooth-Verbindung. Anschließend kann Pi 1 seine eigenen Threads und Ressourcen kontrolliert beenden.

2.8 Programmierrichtlinien und Versionskontrolle

Der C-Code soll MISRA-C-Regeln soweit im Projekt praktikabel berücksichtigen. Insbesondere werden feste Datentypen, geprüfte Rückgabewerte, definierte Wertebereiche, ein `default`-Zweig in `switch`-Anweisungen und möglichst kurze, klar abgegrenzte Funktionen verwendet. Gemeinsame Konstanten wie Timeouts und Grenzwerte werden zentral definiert. Die Entwicklung erfolgt mit Git; Änderungen an Safety-relevanter Logik sollen vor dem Merge von einem zweiten Gruppenmitglied geprüft werden.

3 Programmablauf und Safety-Logik

Abbildung 1 zeigt den Ablauf des Gesamtsystems. Raspberry Pi 1 übernimmt Bedienung, Anzeige, Logging und die zyklische Heartbeat-Kommunikation. Raspberry Pi 2 führt die Sensorerfassung, Safety-Logik und Motorsteuerung aus.

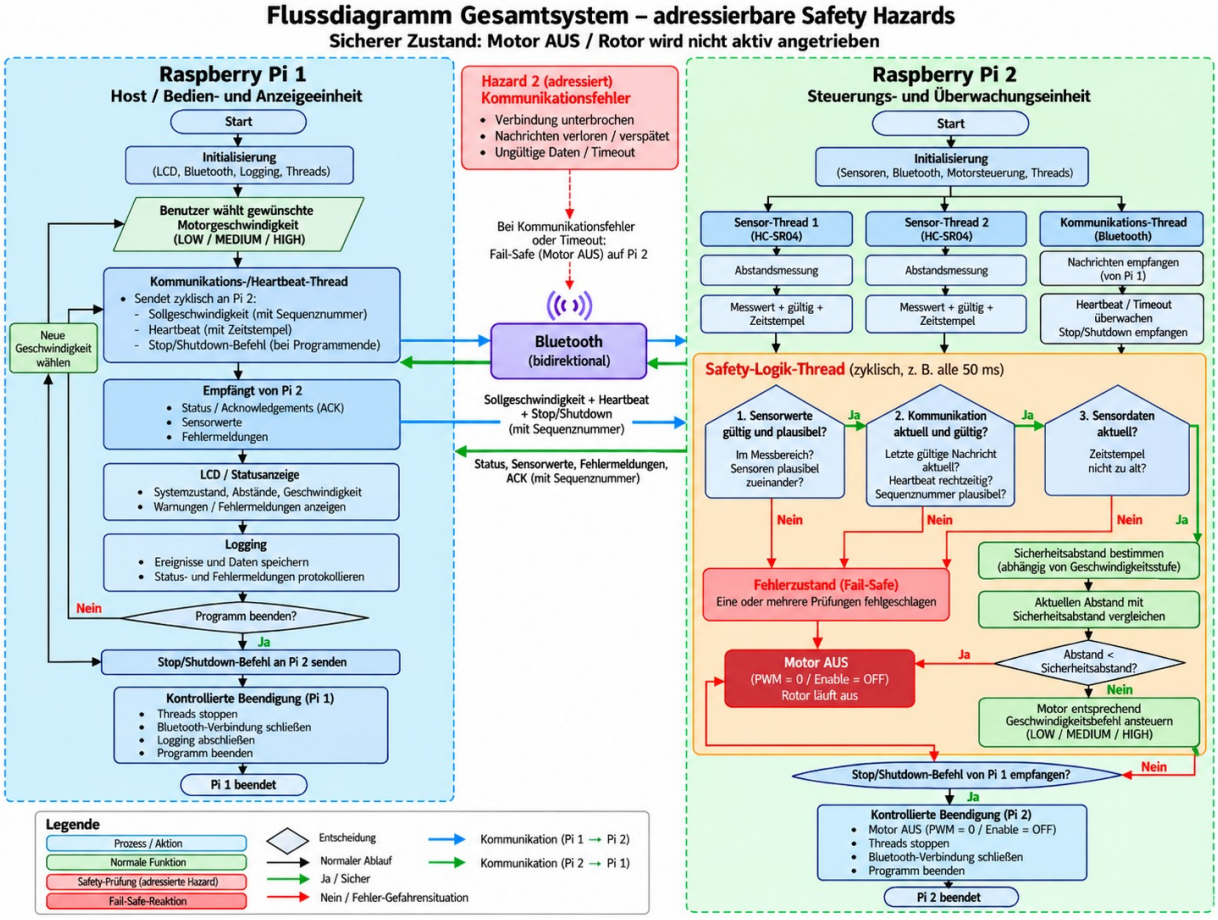


Abbildung 1: Flussdiagramm des Gesamtsystems mit den drei adressierten Safety Hazards

Der Safety-Logik-Thread auf Raspberry Pi 2 arbeitet zyklisch und prüft drei Freigabebedingungen. Die Prüfungen sind logisch UND-verknüpft: Nur wenn alle drei Bedingungen erfüllt sind, wird der Sicherheitsabstand ausgewertet und eine Motorfreigabe ist möglich.

1. **Sensorplausibilität:** Beide Sensorwerte müssen gültig sein. Eine Abweichung von mehr als 5 cm in zwei aufeinanderfolgenden Messzyklen wird als Fehler bewertet.

2. **Kommunikation:** Eine gültige Nachricht bzw. ein Heartbeat muss rechtzeitig eintreffen. Bei mehr als 300 ms ohne gültige Nachricht wird der Motor abgeschaltet.

3. **Datenaktualität:** Sensordaten dürfen höchstens 150 ms alt sein. Ältere Daten gelten als ungültig.

Nach bestandenen Prüfungen wird der Sicherheitsabstand entsprechend der Geschwindigkeitsstufe ausgewählt. Ist der aktuelle Abstand kleiner als der Sicherheitsabstand, wird

der Motor abgeschaltet. Andernfalls darf der Motor mit der freigegebenen Stufe betrieben werden.

Das Programmende ist ebenfalls Teil des Ablaufs. Ein Stop/Shutdown-Befehl von Raspberry Pi 1 führt auf Raspberry Pi 2 zuerst zu Motor AUS und anschließend zum kontrollierten Beenden des Programms.

4 Safety Hazards

Für den Demonstrator werden sechs zentrale Safety Hazards betrachtet. Drei davon werden durch konkrete Maßnahmen im Design adressiert. Drei weitere werden bewusst nicht vollständig adressiert, weil dafür zusätzliche unabhängige oder sicherheitsgerichtete Hardware erforderlich wäre.

Sicherer Zustand: Motor AUS – der Rotor wird nicht aktiv angetrieben.

4.1 Übersicht

ID	Status	Hazard	Umgang im Projekt
H1	Adressiert	Motor läuft weiter, obwohl ein Objekt zu nahe ist, weil ein Sensorwert falsch oder ungültig ist.	Gültigkeitsprüfung, Vergleich beider Sensoren, 5-cm-Plausibilitätsgrenze; bei Fehler Motor AUS.
H2	Adressiert	Motor arbeitet mit einem veralteten oder ungültigen Geschwindigkeitsbefehl weiter.	10-Hz-Heartbeat, Sequenznummer und 300-ms-Timeout; bei Kommunikationsfehler Motor AUS.
H3	Adressiert	Motor läuft weiter, obwohl veraltete Sensordaten verwendet werden.	Zeitstempel je Messwert, maximales Datenalter 150 ms; bei Überschreitung Motor AUS.
H4	Nicht vollständig adressiert	Rotor läuft trotz angefordertem STOP weiter oder kommt zu spät zum Stillstand.	Software setzt PWM = 0 und Enable = OFF; Nachlauf wird berücksichtigt, Defekt der Leistungsstufe bleibt offen.
H5	Nicht vollständig adressiert	Tatsächliche Motordrehzahl ist höher als vom System angenommen.	Keine unabhängige Drehzahlmessung; konservative Geschwindigkeitsklassen und Schutzabstände.
H6	Nicht vollständig adressiert	Beide HC-SR04 liefern gleichzeitig ähnliche falsche Werte.	Quervergleich erkennt Common-Cause-Fehler nicht sicher; Grenze wird dokumentiert.

Tabelle 5: Zentrale Safety Hazards des Demonstrators

4.2 Adressierte Hazards

H1 – Fehlerhafte oder unplausible Abstandsmessung

Hazard: Der Motor läuft weiter, obwohl sich ein Objekt bereits innerhalb des erforderlichen Sicherheitsabstands befindet.

Ursache: Ein Sensor liefert keinen Wert, einen zu großen Wert oder einen deutlich anderen Wert als der zweite Sensor.

Erkennung und Maßnahme: Beide HC-SR04 müssen einen gültigen Messwert liefern. Die absolute Differenz der beiden Werte wird geprüft. Ist

$$|d_1 - d_2| > 5\,\text{cm}$$

in zwei aufeinanderfolgenden Messzyklen, gelten die Sensoren als unplausibel. Ein fehlender oder ungültiger Messwert wird ebenfalls als Fehler behandelt. Für die Abstandsentscheidung wird der kleinere der beiden gültigen Werte verwendet.

Fail-Safe-Reaktion:

Ungültige oder unplausible Sensordaten → Motor AUS

Test: Einen Sensor verdecken, gezielt einen abweichenden Messwert erzeugen oder per Testmodus einspeisen. Der Fehler muss erkannt werden und der Motor muss abschalten.

H2 – Kommunikationsfehler zwischen Raspberry Pi 1 und Raspberry Pi 2

Hazard: Der Motor arbeitet mit einem alten oder ungültigen Geschwindigkeitsbefehl weiter.

Ursache: Bluetooth-Verbindung unterbrochen, Pi 1 ausgefallen, Nachricht verspätet oder Nachrichteninhalt ungültig.

Erkennung und Maßnahme: Pi 1 sendet alle 100 ms einen Heartbeat zusammen mit dem aktuellen Befehl. Pi 2 prüft Wertebereich und Sequenznummer. Trifft länger als 300 ms keine gültige Nachricht ein, wird ein Kommunikationsfehler gesetzt. Damit entsprechen drei ausbleibende Heartbeats dem Timeout.

Fail-Safe-Reaktion:

Kommunikationsfehler oder Timeout → Motor AUS

Test: Während des Motorbetriebs die Bluetooth-Verbindung trennen oder das Pi-1-Programm anhalten. Spätestens nach dem definierten Timeout muss Pi 2 den Motor abschalten.

H3 – Veraltete Sensordaten

Hazard: Die Safety-Entscheidung wird mit einem Abstandswert getroffen, der nicht mehr den aktuellen Zustand beschreibt.

Ursache: Ein Sensor-Thread hängt, wird verspätet ausgeführt oder aktualisiert seine Daten nicht mehr.

Erkennung und Maßnahme: Jeder Messwert wird mit einem monotonen Zeitstempel gespeichert. Vor der Verwendung prüft der Safety-Logik-Thread das Alter. Messwerte älter als 150 ms gelten als ungültig und dürfen keine Motorfreigabe bewirken.

Fail-Safe-Reaktion:

Sensordaten zu alt → Motor AUS

Test: Einen Sensor-Thread im Testmodus länger als 150 ms blockieren. Der Safety-Thread muss den veralteten Messwert erkennen und den Motor abschalten.

4.3 Nicht vollständig adressierte Hazards

H4 – Motor oder Motortreiber reagiert nicht korrekt auf STOP

Hazard: Der Rotor wird trotz angeforderter Abschaltung weiter angetrieben oder kommt erst verspätet zum Stillstand.

Warum relevant: Ein Software-Stoppkommando garantiert weder eine funktionierende Leistungselektronik noch sofortigen mechanischen Stillstand.

Warum nicht vollständig adressiert: Die vorhandene Architektur besitzt keinen unabhängigen sicherheitsgerichteten Abschaltpfad und keine Rückmeldung, ob der Rotor tatsächlich steht.

Maßnahme im Demonstrator: PWM und Enable werden deaktiviert. Die Nachlaufzeit des Rotors wird am realen Aufbau gemessen und bei der Wahl der Sicherheitsabstände berücksichtigt.

Industrielle Lösung: Unabhängige sicherheitsgerichtete Abschaltung, beispielsweise geeigneter Safety-Controller, Sicherheitsrelais oder ein Antrieb mit Safe-Torque-Off.

H5 – Tatsächliche Motordrehzahl ist unbekannt

Hazard: Der Rotor dreht schneller, als es die gewählte Geschwindigkeitsstufe vermuten lässt.

Warum relevant: PWM-Prozent und reale Motordrehzahl sind nicht gleichzusetzen. Last, Versorgungsspannung und Motoreigenschaften beeinflussen die tatsächliche Drehzahl und Nachlaufzeit.

Warum nicht vollständig adressiert: Es ist kein unabhängiger Encoder oder anderer Drehzahlsensor vorhanden.

Maßnahme im Demonstrator: Es werden nur die Klassen `LOW`, `MEDIUM` und `HIGH` verwendet. Für höhere Stufen wird ein größerer, konservativ festgelegter Sicherheitsabstand verwendet.

Industrielle Lösung: Unabhängige Drehzahlerfassung mit sicherheitsgerichteter Auswertung.

H6 – Gemeinsamer Fehler beider Abstandssensoren

Hazard: Beide HC-SR04 liefern gleichzeitig ähnliche, aber falsche Abstandswerte.

Warum relevant: Beide Sensoren verwenden dasselbe Messprinzip. Ungünstige Oberflächen, Winkel, Reflexionen oder gemeinsame Störungen können daher beide Sensoren ähnlich beeinflussen.

Warum nicht vollständig adressiert: Ein Vergleich zweier gleicher Sensoren erkennt nur Unterschiede zwischen den Messwerten. Sind beide Werte durch dieselbe Ursache falsch, kann die Plausibilitätsprüfung erfolgreich sein, obwohl der reale Abstand gefährlich klein ist.

Maßnahme im Demonstrator: Die Sensoren werden zeitlich versetzt angesteuert und die Einschränkung des Quervergleichs wird ausdrücklich dokumentiert.

Industrielle Lösung: Geeignete sicherheitsgerichtete Sensorik und je nach Anwendung diversitäre Messprinzipien.

5 Security-Betrachtung

Die Security-Betrachtung konzentriert sich auf die Bluetooth-Kommunikation zwischen den beiden Raspberry Pis. Die lokale Abstandsauswertung auf Raspberry Pi 2 bleibt auch dann aktiv, wenn die Funkverbindung gestört oder unterbrochen wird. Security-Maßnahmen dürfen daher nicht die Fail-Safe-Reaktion verhindern.

5.1 Relevante Bedrohungen

ID	Bedrohung	Mögliche Auswirkung	Maßnahme im Demonstrator
T1	Unberechtigtes Gerät sendet Befehle	Falsche Geschwindigkeitsstufe oder unerwünschte Bedienung	Bluetooth-Pairing/Bonding; Pi 2 akzeptiert nur den vorgesehenen Kommunikationspartner; Nachrichteninhalt wird geprüft.
T2	Alte oder doppelte Nachricht wird erneut verarbeitet	Veralteter Befehl könnte erneut wirksam werden	Sequenznummer je Nachricht; alte und doppelte Nachrichten werden verworfen.
T3	Bluetooth wird gestört oder fällt aus	Pi 2 erhält keine aktuellen Befehle mehr	Heartbeat und 300-ms-Timeout; bei Kommunikationsverlust Motor AUS. Verfügbarkeit kann dadurch verloren gehen, Safety bleibt erhalten.

Tabelle 6: Security-Bedrohungen und Gegenmaßnahmen

5.2 Abgrenzung

Der Demonstrator implementiert kein eigenes kryptographisches Protokoll und keinen vollständigen Schutz gegen einen kompromittierten Raspberry Pi oder gezieltes Funk-Jamming. Ein Angreifer mit physischem Zugriff auf das Gerät oder auf das Betriebssystem liegt außerhalb des Projektumfangs. Entscheidend für das Safety-Konzept ist, dass fehlerhafte oder ausbleibende Kommunikation keine Motorfreigabe aufrechterhält.

6 Anforderungen und Tests

Die Anforderungen sind bewusst auf die für den Demonstrator wesentlichen Punkte beschränkt. Zeit- und Grenzwerte sind Design-Startwerte und müssen am realen Aufbau verifiziert werden.

6.1 Safety-Anforderungen

ID	Anforderung	Bezug
SAF-01	Der sichere Zustand des Systems ist Motor AUS. Auf Pi 2 bedeutet dies PWM = 0 und Enable = OFF.	H1–H6
SAF-02	Pi 2 muss beim Programmstart im sicheren Zustand bleiben, bis Initialisierung, Sensorprüfung und Kommunikation gültig sind.	H1–H3
SAF-03	Beide Abstandssensoren müssen gültige Werte liefern. Eine Abweichung größer 5 cm in zwei aufeinanderfolgenden Messzyklen muss zu Motor AUS führen.	H1
SAF-04	Jeder Sensormesswert muss einen Zeitstempel besitzen. Daten älter als 150 ms dürfen keine Motorfreigabe bewirken.	H3
SAF-05	Pi 1 muss alle 100 ms einen Heartbeat senden. Erhält Pi 2 länger als 300 ms keine gültige Nachricht, muss der Motor abgeschaltet werden.	H2
SAF-06	Der Safety-Logik-Thread auf Pi 2 soll zyklisch alle 50 ms laufen und die Safety-Bedingungen vor einer Motorfreigabe prüfen.	H1–H3
SAF-07	Nur die gültigen Geschwindigkeitsstufen LOW, MEDIUM und HIGH dürfen verwendet werden. Für jede Stufe muss ein definierter Sicherheitsabstand existieren.	H5
SAF-08	Unterschreitet der aktuelle gültige Abstand den Sicherheitsabstand der gewählten Stufe, muss der Motor abgeschaltet werden.	H1, H5
SAF-09	Ein Stop/Shutdown-Befehl muss auf Pi 2 zuerst Motor AUS auslösen. Erst danach dürfen Threads und Programm beendet werden.	H4
SAF-10	Die endgültigen Sicherheitsabstände müssen am realen Aufbau anhand der beobachteten Reaktions- und Nachlaufzeiten festgelegt und dokumentiert werden.	H4, H5

Tabelle 7: Safety-Anforderungen

6.2 Security-Anforderungen

ID	Anforderung
SEC-01	Die Bluetooth-Verbindung soll nur mit dem vorgesehenen, gepaarten bzw. gebondeten Kommunikationspartner aufgebaut werden.
SEC-02	Eingehende Nachrichten müssen vor der Verwendung auf Typ, Länge und gültigen Wertebereich geprüft werden.
SEC-03	Jede Steuerungsnachricht muss eine Sequenznummer tragen. Alte oder doppelte Nachrichten dürfen nicht erneut ausgeführt werden.
SEC-04	Ein Kommunikationsangriff oder Kommunikationsausfall darf die lokale Abstandsauswertung auf Pi 2 nicht deaktivieren; bei Timeout gilt Motor AUS.

Tabelle 8: Security-Anforderungen

6.3 Unterstützende Anforderungen

- Das LCD zeigt Systemzustand, Abstände, Geschwindigkeitsstufe und Fehler an. Das LCD ist keine Safety-Funktion.

- Logging protokolliert wichtige Zustands- und Fehlerereignisse. Logging ist keine Risikominderung und darf den Safety-Logik-Thread nicht blockieren.
- Gemeinsame Daten zwischen Threads müssen konsistent übertragen werden, beispielsweise über Mutex und Snapshot-Kopie.
- Änderungen an Safety-relevanter Software werden über Git versioniert und sollen vor dem Merge überprüft werden.

6.4 Geplante Tests

Test	**Durchführung**	**Erwartetes Ergebnis**
TC-01	Einen Sensor verdecken oder ungültigen Messwert erzeugen.	Sensorfehler wird erkannt; Motor AUS.
TC-02	Zwei Sensorwerte um mehr als 5 cm abweichen lassen.	Nach zwei aufeinanderfolgenden unplausiblen Messzyklen Motor AUS.
TC-03	Bluetooth-Verbindung während des Motorbetriebs trennen oder Pi 1 anhalten.	Spätestens nach 300 ms ohne gültige Nachricht wird die Motorfreigabe entzogen.
TC-04	Sensor-Thread länger als 150 ms blockieren.	Daten werden als veraltet erkannt; Motor AUS.
TC-05	Testobjekt in den Sicherheitsabstand der jeweiligen Geschwindigkeitsstufe bewegen.	Motor wird abgeschaltet, sobald der gültige Abstand kleiner als der konfigurierte Sicherheitsabstand ist.
TC-06	Programmende auf Pi 1 auslösen.	Pi 1 sendet Stop/Shutdown; Pi 2 setzt zuerst Motor AUS und beide Programme beenden kontrolliert.

Tabelle 9: Geplante System- und Safety-Tests